

DSM – 2º Semestre (Banco de Dados)

Integrantes: Cid Daniel e Arthur Óliver

Prática 01

PRÁTICA 1

ARTHUR ÓLIVER ROSSI ALVES

CID DANIEL NEVES DOLIVEIRA

USE VENDA2;

#EXERCÍCIO 1

-- Inserir pelo menos mais dez linhas em cada tabela.

-- CLIENTES

INSERT INTO cliente VALUES

(41,'Lucas Souza','12345678941','Rua das Palmeiras','São Paulo','01000-001','SP','Centro');

INSERT INTO cliente VALUES

(42,'Mariana Oliveira','12345678942','Rua das Acácias','São Paulo','01000-002','SP','Moema');

INSERT INTO cliente VALUES

(43,'Pedro Santos','12345678943','Rua das Rosas','São Paulo','01000-003','SP','Lapa');

INSERT INTO cliente VALUES

(44,'Camila Souza','12345678944','Rua das Orquídeas','São Paulo','01000-004','SP','Centro');

INSERT INTO cliente VALUES

(45,'Rafael Costa','12345678945','Rua das Flores','Rio de Janeiro','20000-001','RJ','Copacabana');

INSERT INTO cliente VALUES

(46,'Beatriz Lima','12345678946','Rua do Sol','Rio de Janeiro','20000-002','RJ','Ipanema');

INSERT INTO cliente VALUES

(47,'Gabriel Mendes','12345678947','Rua do Mar','Rio de Janeiro','20000-003','RJ','Botafogo');

INSERT INTO cliente VALUES

(48,'Larissa Souza','12345678948','Rua Central','São Paulo','01000-005','SP','Centro');

INSERT INTO cliente VALUES

(49,'Mateus Ferreira','12345678949','Rua Brasil','São Paulo','01000-006','SP','Santana');

INSERT INTO cliente VALUES

(50,'Isabela Rocha','12345678950','Rua da Paz','Rio de Janeiro','20000-004','RJ','Tijuca');

-- PRODUTOS

INSERT INTO produto VALUES

(16,'Biscoito Cream Cracker',6.50,100);

INSERT INTO produto VALUES

(17,'Achocolatado 400g',9.90,80);

INSERT INTO produto VALUES

(18,'Farinha de Trigo 1kg',5.90,90);

INSERT INTO produto VALUES

(19,'Molho de Tomate 340g',3.50,120);

INSERT INTO produto VALUES

(20,'Aveia em Flocos 500g',8.90,70);

INSERT INTO produto VALUES

(21,'Maionese 500g',12.50,60);

INSERT INTO produto VALUES

(22,'Ketchup 400g',7.90,60);

INSERT INTO produto VALUES

(23,'Mostarda 200g',5.50,50);

INSERT INTO produto VALUES

(24,'Granola 500g',15.90,40);

INSERT INTO produto VALUES

(25,'Cereal 300g',13.90,45);

-- VENDEDORES

INSERT INTO vendedor VALUES

```
(6,'Marcos Vendas',2300);
```

```
INSERT INTO vendedor VALUES
```

```
(7,'Amanda Vendas',2400);
```

```
INSERT INTO vendedor VALUES
```

```
(8,'Felipe Vendas',1900);
```

```
INSERT INTO vendedor VALUES
```

```
(9,'Beatriz Vendas',2600);
```

```
INSERT INTO vendedor VALUES
```

```
(10,'Lucas Vendas',2200);
```

```
INSERT INTO vendedor VALUES
```

```
(11,'Gabriela Vendas',2500);
```

```
INSERT INTO vendedor VALUES
```

```
(12,'Daniel Vendas',2100);
```

```
INSERT INTO vendedor VALUES
```

```
(13,'Renata Vendas',2300);
```

```
INSERT INTO vendedor VALUES
```

```
(14,'Thiago Vendas',2000);
```

```
INSERT INTO vendedor VALUES
```

```
(15,'Larissa Vendas',2700);
```

-- PEDIDOS

INSERT INTO pedido VALUES

(26,'2026-05-31',41,6,NULL,4);

INSERT INTO pedido VALUES

(27,'2026-05-31',42,7,NULL,10);

INSERT INTO pedido VALUES

(28,'2026-05-31',43,8,NULL,20);

INSERT INTO pedido VALUES

(29,'2026-05-31',44,9,NULL,18);

INSERT INTO pedido VALUES

(30,'2026-06-01',45,10,NULL,20);

INSERT INTO pedido VALUES

(31,'2026-06-02',46,11,NULL,25);

INSERT INTO pedido VALUES

(32,'2026-06-03',47,12,NULL,16);

INSERT INTO pedido VALUES

(33,'2026-06-04',48,13,NULL,5);

```
INSERT INTO pedido VALUES
```

```
(34,'2026-06-05',49,14,NULL,8);
```

```
INSERT INTO pedido VALUES
```

```
(35,'2026-06-06',50,15,NULL,30);
```

```
-- ITEM_PEDIDO
```

```
INSERT INTO item_pedido VALUES
```

```
(26,16,2,13.00);
```

```
INSERT INTO item_pedido VALUES
```

```
(27,17,3,29.70);
```

```
INSERT INTO item_pedido VALUES
```

```
(28,18,2,11.80);
```

```
INSERT INTO item_pedido VALUES
```

```
(29,19,4,14.00);
```

```
INSERT INTO item_pedido VALUES
```

```
(30,20,2,17.80);
```

```
INSERT INTO item_pedido VALUES
```

```
(31,16,5,32.50);
```

```
INSERT INTO item_pedido VALUES
```

(32,17,6,59.40);

INSERT INTO item_pedido VALUES

(33,18,3,17.70);

INSERT INTO item_pedido VALUES

(34,19,8,28.00);

INSERT INTO item_pedido VALUES

(35,20,7,62.30);

-- PAGAMENTOS

INSERT INTO pagamento VALUES

(36,26,'Pix',13.00);

INSERT INTO pagamento VALUES

(37,27,'Cartao',29.70);

INSERT INTO pagamento VALUES

(38,28,'Pix',11.80);

INSERT INTO pagamento VALUES

(39,29,'Cartao',14.00);

INSERT INTO pagamento VALUES

(40,30,'Pix',17.80);

INSERT INTO pagamento VALUES

(41,31,'Cartao',32.50);

INSERT INTO pagamento VALUES

(42,32,'Pix',59.40);

INSERT INTO pagamento VALUES

(43,33,'Cartao',17.70);

INSERT INTO pagamento VALUES

(44,34,'Pix',28.00);

INSERT INTO pagamento VALUES

(45,35,'Cartao',62.30);

-- TELEFONES

INSERT INTO telefone VALUES

(1,41,'(11)99999-0041','Celular');

INSERT INTO telefone VALUES

(1,42,'(11)99999-0042','Celular');

INSERT INTO telefone VALUES

(1,43,'(11)99999-0043','Celular');

INSERT INTO telefone VALUES

(1,44,'(11)99999-0044','Celular');

INSERT INTO telefone VALUES

(1,45,'(21)99999-0045','Celular');

INSERT INTO telefone VALUES

(1,46,'(21)99999-0046','Celular');

INSERT INTO telefone VALUES

(1,47,'(21)99999-0047','Celular');

INSERT INTO telefone VALUES

(1,48,'(11)99999-0048','Celular');

INSERT INTO telefone VALUES

(1,49,'(11)99999-0049','Celular');

INSERT INTO telefone VALUES

(1,50,'(21)99999-0050','Celular');

Output		
History Output		
Date	Time	SQL
2026-08-27	12:02:55	INSERT INTO telefone VALUES (1,50,'(21)99999-0050','Celular')
2026-08-24	12:02:54	INSERT INTO telefone VALUES (1,49,'(11)99999-0049','Celular')
2026-08-20	12:02:54	INSERT INTO telefone VALUES (1,48,'(11)99999-0048','Celular')
2025-06-12	12:02:54	INSERT INTO telefone VALUES (1,47,'(21)99999-0047','Celular')
2025-06-05	12:02:54	INSERT INTO telefone VALUES (1,46,'(21)99999-0046','Celular')
2025-06-02	12:02:54	INSERT INTO telefone VALUES (1,45,'(21)99999-0045','Celular')
2025-05-29	12:02:54	INSERT INTO telefone VALUES (1,44,'(11)99999-0044','Celular')
2025-05-15	12:02:54	INSERT INTO telefone VALUES (1,43,'(11)99999-0043','Celular')
2025-05-08	12:02:54	INSERT INTO telefone VALUES (1,42,'(11)99999-0042','Celular')
2025-05-05	12:02:54	INSERT INTO telefone VALUES (1,41,'(11)99999-0041','Celular')
2025-04-30	12:02:54	INSERT INTO pagamento VALUES (45,35,'Cartao',62.30)
2025-04-28	12:02:54	INSERT INTO pagamento VALUES (44,34,'Pix',28.00)
2025-04-27	12:02:54	INSERT INTO pagamento VALUES (43,33,'Cartao',17.70)
2025-04-26	12:02:54	INSERT INTO pagamento VALUES (42,32,'Pix',59.40)
2025-04-25	12:02:54	INSERT INTO pagamento VALUES (41,31,'Cartao',32.50)
2025-04-24	12:02:54	INSERT INTO pagamento VALUES (40,30,'Pix',17.80)
2025-03-20	12:02:54	INSERT INTO pagamento VALUES (39,29,'Cartao',14.00)
2025-03-13	12:02:54	INSERT INTO pagamento VALUES (38,28,'Pix',11.80)
	12:02:54	INSERT INTO pagamento VALUES (37,27,'Cartao',29.70)
	12:02:54	INSERT INTO pagamento VALUES (36,26,'Pix',13.00)
	12:02:54	INSERT INTO item_pedido VALUES (35,20,7.62.30)
	12:02:54	INSERT INTO item_pedido VALUES (34,19,8.28.00)
	12:02:54	INSERT INTO item_pedido VALUES (33,18,3.17.70)
	12:02:54	INSERT INTO item_pedido VALUES (32,17,6.59.40)
	12:02:54	INSERT INTO item_pedido VALUES (31,16,5.32.50)
	12:02:54	INSERT INTO item_pedido VALUES (30,20,2.17.80)
	12:02:54	INSERT INTO item_pedido VALUES (29,19,4.14.00)

#EXERCÍCIO 2

-- Listar todos os clientes, ordenados por nome do cliente.

SELECT *

FROM cliente

ORDER BY cli_nome;

Result Grid		Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:	
	cli_codigo	cli_nome	cli_cpf	cli_logradouro	cli_cidade	cli_cep	cli_estado	cli_bairro	
▶	15	Aline Castro	12345678915	Rua Padre Almeida	Campinas	13025-251	SP	Cambuí	
	3	Ana Paula Costa	12345678903	Rua Barreto Leme	Campinas	13013-000	SP	Centro	
	12	André Luiz Barbosa	12345678912	Av Orosimbo Maia	Campinas	13010-906	SP	Centro	
	46	Beatriz Lima	12345678946	Rua do Sol	Rio de Janeiro	20000-002	RJ	Ipanema	
	33	Beatriz Tavares	12345678933	Rua General Osório	Campinas	13010-110	SP	Centro	
	14	Bruno Teixeira	12345678914	Rua Dr Quirino	Campinas	13015-081	SP	Centro	
	13	Camila Freitas	12345678913	Rua Culto à Ciência	Campinas	13020-060	SP	Botafogo	
	44	Camila Souza	12345678944	Rua das Orquídeas	São Paulo	01000-004	SP	Centro	
	31	Carla Nogueira	12345678931	Rua Bernardino de Campos	Campinas	13010-151	SP	Centro	
	4	Carlos Eduardo Lima	12345678904	Rua Conceição	Campinas	13015-000	SP	Cambuí	
	39	Cristiane Lopes	12345678939	Rua Barreto Leme	Campinas	13010-300	SP	Centro	
	21	Daniela Souza	12345678921	Rua Barreto Leme	Campinas	13010-200	SP	Centro	
	35	Débora Rezende	12345678935	Rua Conceição	Campinas	13015-100	SP	Centro	
	40	Diego Freitas	12345678940	Rua Carolina Florence	Campinas	13087-000	SP	Barão G...	

#EXERCÍCIO 3

-- Listar os clientes que os sobrenomes possuem 'Souza' e são do estado de São Paulo

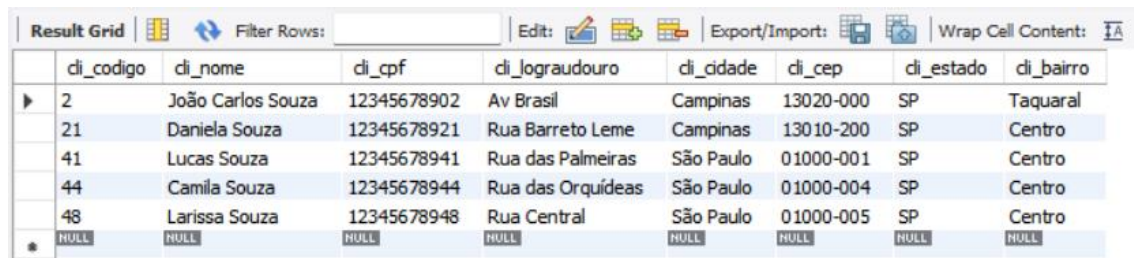
-- (SP). Dica: Usar o comando like.

```
SELECT *
```

```
FROM cliente
```

```
WHERE cli_nome LIKE '%Souza%'
```

```
AND cli_estado = 'SP';
```



The screenshot shows a database query result grid with the following columns: cli_codigo, cli_nome, cli_cpf, cli_logradouro, cli_cidade, cli_cep, cli_estado, and cli_bairro. The results are as follows:

cli_codigo	cli_nome	cli_cpf	cli_logradouro	cli_cidade	cli_cep	cli_estado	cli_bairro
2	João Carlos Souza	12345678902	Av Brasil	Campinas	13020-000	SP	Taquaral
21	Daniela Souza	12345678921	Rua Barreto Leme	Campinas	13010-200	SP	Centro
41	Lucas Souza	12345678941	Rua das Palmeiras	São Paulo	01000-001	SP	Centro
44	Camila Souza	12345678944	Rua das Orquideas	São Paulo	01000-004	SP	Centro
48	Larissa Souza	12345678948	Rua Central	São Paulo	01000-005	SP	Centro
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

#EXERCÍCIO 4

-- Listar o nome do cliente do pedido de número 1.

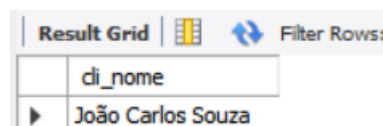
```
SELECT c.cli_nome
```

```
FROM cliente c
```

```
INNER JOIN pedido p
```

```
ON c.cli_codigo = p.cli_codigo
```

```
WHERE p.ped_numero = 1;
```



The screenshot shows a database query result grid with the following columns: cli_nome. The result is as follows:

cli_nome
João Carlos Souza

#EXERCÍCIO 5

-- Listar o nome do cliente, o nome do vendedor do pedido de código 2.

```

SELECT c.cli_nome, v.ven_nome

FROM pedido p

INNER JOIN cliente c

ON p.cli_codigo = c.cli_codigo

INNER JOIN vendedor v

ON p.ven_codigo = v.ven_codigo

WHERE p.ped_numero = 2;

```

Result Grid			Filter Rows:
	cli_nome	ven_nome	
▶	João Carlos Souza	Juliana Vendas	

#EXERCÍCIO 6

-- Listar a quantidade de pedido.

```

SELECT COUNT(*) AS quantidade_pedidos

FROM pedido;

```

Result Grid		Filter Rows:
	quantidade_pedidos	
▶	35	

#EXERCÍCIO 7

-- Listar a quantidade de pedido que cada vendedor realizou (Dica: Utilizar group by).

```

SELECT v.ven_nome, COUNT(p.ped_numero) AS quantidade_pedidos

FROM vendedor v

INNER JOIN pedido p

ON v.ven_codigo = p.ven_codigo

```

GROUP BY v.ven_codigo, v.ven_nome;

Result Grid   Filter Rows:

	ven_nome	quantidade_pedidos
▶	Carlos Vendas	5
	Juliana Vendas	5
	Roberto Vendas	5
	Paula Vendas	5
	Ricardo Vendas	5
	Marcos Vendas	1
	Amanda Vendas	1
	Felipe Vendas	1
	Beatriz Vendas	1
	Lucas Vendas	1
	Gabriela Vendas	1
	Daniel Vendas	1
	Renata Vendas	1
	Thiago Vendas	1

#EXERCÍCIO 8

-- Listar todos os itens do pedido de código 1.

SELECT *

FROM item_pedido

WHERE ped_numero = 1;

Result Grid			 Filter Rows:		Edit:			
	ped_numero	pro_codigo	item_quantidade	item_valor				
▶	1	1	1	51.00				
	1	2	1	52.00				
*	NULL	NULL	NULL	NULL				

#EXERCÍCIO 9

-- Listar a soma total do pedido de código 1.

SELECT SUM(item_valor) AS total_pedido

FROM item_pedido

WHERE ped_numero = 1;

Result Grid	
	total_pedido
▶	103.00

#EXERCÍCIO 10

-- Listar os clientes e os seus telefones (Listar apenas os clientes que possuam telefone).

-- Dica: Junção: Inner join /where.

```
SELECT c.cli_nome, t.tel_numero
```

```
FROM cliente c
```

```
INNER JOIN telefone t
```

```
ON c.cli_codigo = t.cli_codigo;
```

Result Grid		
	Filter Rows:	
	cli_nome	tel_numero
▶	Maria Aparecida Silva	(19)99999-0001
	João Carlos Souza	(19)99999-0002
	Ana Paula Costa	(19)99999-0003
	Carlos Eduardo Lima	(19)99999-0004
	Fernanda Alves Rocha	(19)99999-0005
	Ricardo Gomes Pereira	(19)99999-0006
	Juliana Martins	(19)99999-0007
	Paulo Henrique Dias	(19)99999-0008
	Patrícia Fernandes	(19)99999-0009
	Roberto Nunes	(19)99999-0010
	Luciana Ribeiro	(19)99999-0011
	André Luiz Barbosa	(19)99999-0012
	Camila Freitas	(19)99999-0013
	Bruno Teixeira	(19)99999-0014

#EXERCÍCIO 11

-- Listar os clientes e os seus telefones (Listar também os clientes que não tenham

-- telefone).

```
SELECT c.cli_nome, t.tel_numero
```

```
FROM cliente c
```

LEFT JOIN telefone t

ON c.cli_codigo = t.cli_codigo;

Result Grid		Filter Rows:
	cli_nome	tel_numero
▶	Maria Aparecida Silva	(19)99999-0001
	João Carlos Souza	(19)99999-0002
	Ana Paula Costa	(19)99999-0003
	Carlos Eduardo Lima	(19)99999-0004
	Fernanda Alves Rocha	(19)99999-0005
	Ricardo Gomes Pereira	(19)99999-0006
	Juliana Martins	(19)99999-0007
	Paulo Henrique Dias	(19)99999-0008
	Patrícia Fernandes	(19)99999-0009
	Roberto Nunes	(19)99999-0010
	Luciana Ribeiro	(19)99999-0011
	André Luiz Barbosa	(19)99999-0012
	Camila Freitas	(19)99999-0013
	Bruno Teixeira	(19)99999-0014

#EXERCÍCIO 12

-- Listar todos os pedidos realizados na data 31/05/2026.

SELECT *

FROM pedido

WHERE ped_data = '2026-05-31';

Result Grid

Filter Rows:

Edit:

Export/Import:

	ped_numero	ped_data	cli_codigo	ven_codigo	ped_valor_total	ped_prazo_entrega
▶	26	2026-05-31	41	6	NULL	4
	27	2026-05-31	42	7	NULL	10
	28	2026-05-31	43	8	NULL	20
	29	2026-05-31	44	9	NULL	18
*	NULL	NULL	NULL	NULL	NULL	NULL

#EXERCÍCIO 13

-- Listar todos os pedidos realizados na data 31/05/2026 e que possuem prazo de entrega

-- maior que 3.

```

SELECT *

FROM pedido

WHERE ped_data = '2026-05-31'

AND ped_prazo_entrega > 3;

```

	ped_numero	ped_data	cli_codigo	ven_codigo	ped_valor_total	ped_prazo_entrega
▶	26	2026-05-31	41	6	NULL	4
	27	2026-05-31	42	7	NULL	10
	28	2026-05-31	43	8	NULL	20
	29	2026-05-31	44	9	NULL	18
*	NULL	NULL	NULL	NULL	NULL	NULL

#EXERCÍCIO 14

-- Listar todas as vendas(pedidos) do produto Leite Integral 1L.

```

SELECT DISTINCT p.ped_numero

FROM pedido p

INNER JOIN item_pedido i

ON p.ped_numero = i.ped_numero

INNER JOIN produto pr

ON i.pro_codigo = pr.pro_codigo

WHERE pr.pro_descricao = 'Leite Integral 1L';

```

	ped_numero
▶	4
	11
	19

#EXERCÍCIO 15

-- Listar todos os pedidos que o cliente de código 1 realizou (Listar o número do pedido,
-- data do pedido e o nome do cliente).

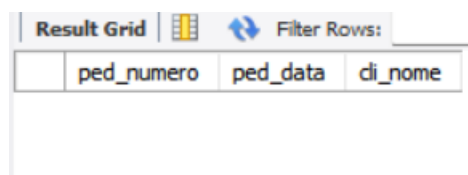
```
SELECT p.ped_numero, p.ped_data, c.cli_nome
```

```
FROM pedido p
```

```
INNER JOIN cliente c
```

```
ON p.cli_codigo = c.cli_codigo
```

```
WHERE c.cli_codigo = 1;
```



ped_numero	ped_data	cli_nome
------------	----------	----------

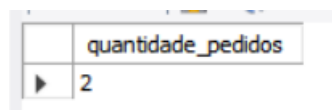
#EXERCÍCIO 16

-- Listar a quantidade de pedido que o cliente do código 2 realizou.

```
SELECT COUNT(*) AS quantidade_pedidos
```

```
FROM pedido
```

```
WHERE cli_codigo = 2;
```



quantidade_pedidos
2

#EXERCÍCIO 17

-- Listar o nome do produto de maior valor.

```
SELECT pro_descricao, pro_valor_unidade
```

```
FROM produto
```

```
WHERE pro_valor_unidade = (
```

```

SELECT MAX(pro_valor_unidade)

FROM produto

);

```

Result Grid			Filter Rows:
	pro_descricao	pro_valor_unidade	
▶	Queijo Mussarela 1kg	39.90	

#EXERCÍCIO 18

-- Quais os pedidos que têm quantidade de itens maior que 5?

```

SELECT ped_numero

FROM item_pedido

GROUP BY ped_numero

HAVING COUNT(*) > 5;

```

Result Grid		Filter Rows:
	ped_numero	

#EXERCÍCIO 19

-- Quais clientes têm pedidos com prazo de entrega maior que 15 dias e que são do estado de
 -- São Paulo ou Rio de Janeiro?

```

SELECT DISTINCT c.cli_nome, c.cli_estado, p.ped_numero, p.ped_prazo_entrega

FROM cliente c

INNER JOIN pedido p

ON c.cli_codigo = p.cli_codigo

WHERE p.ped_prazo_entrega > 15

```

AND c.cli_estado IN ('SP', 'RJ');

Result Grid	Filter Rows:	Export:	Wrap Ce
cli_nome	cli_estado	ped_numero	ped_prazo_entrega
Pedro Santos	SP	28	20
Camila Souza	SP	29	18
Rafael Costa	RJ	30	20
Beatriz Lima	RJ	31	25
Gabriel Mendes	RJ	32	16
Isabela Rocha	RJ	35	30

#EXERCÍCIO 20

-- Liste o número do pedido, a data do pedido e o nome do cliente.

SELECT p.ped_numero, p.ped_data, c.cli_nome

FROM pedido p

INNER JOIN cliente c

ON p.cli_codigo = c.cli_codigo;

Result Grid	Filter Rows:	E
ped_numero	ped_data	cli_nome
1	2026-08-27	João Carlos Souza
2	2026-08-27	João Carlos Souza
3	2026-08-27	Ana Paula Costa
4	2026-08-27	Carlos Eduardo Lima
5	2026-08-27	Fernanda Alves Rocha
6	2026-08-27	Ricardo Gomes Pereira
7	2026-08-27	Juliana Martins
8	2026-08-27	Paulo Henrique Dias
9	2026-08-27	Patrícia Fernandes
10	2026-08-27	Roberto Nunes
11	2026-08-27	Luciana Ribeiro
12	2026-08-27	André Luiz Barbosa
13	2026-08-27	Camila Freitas
14	2026-08-27	Bruno Teixeira

#EXERCÍCIO 21

-- Liste o número do pedido, o nome do vendedor e a data do pedido.

```
SELECT p.ped_numero, v.ven_nome, p.ped_data
```

```
FROM pedido p
```

```
INNER JOIN vendedor v
```

```
ON p.ven_codigo = v.ven_codigo;
```

Result Grid   Filter Rows:			
	ped_numero	ven_nome	ped_data
▶	1	Carlos Vendas	2026-08-27
	6	Carlos Vendas	2026-08-27
	11	Carlos Vendas	2026-08-27
	16	Carlos Vendas	2026-08-27
	21	Carlos Vendas	2026-08-27
	2	Juliana Vendas	2026-08-27
	7	Juliana Vendas	2026-08-27
	12	Juliana Vendas	2026-08-27
	17	Juliana Vendas	2026-08-27
	22	Juliana Vendas	2026-08-27
	3	Roberto Vendas	2026-08-27
	8	Roberto Vendas	2026-08-27
	13	Roberto Vendas	2026-08-27
	18	Roberto Vendas	2026-08-27

#EXERCÍCIO 22

-- Liste o nome do cliente e seu telefone.

```
SELECT c.cli_nome, t.tel_numero
```

```
FROM cliente c
```

```
INNER JOIN telefone t
```

```
ON c.cli_codigo = t.cli_codigo;
```

Result Grid			Filter Rows:
	cli_nome	tel_numero	
▶	Maria Aparecida Silva	(19)99999-0001	
	João Carlos Souza	(19)99999-0002	
	Ana Paula Costa	(19)99999-0003	
	Carlos Eduardo Lima	(19)99999-0004	
	Fernanda Alves Rocha	(19)99999-0005	
	Ricardo Gomes Pereira	(19)99999-0006	
	Juliana Martins	(19)99999-0007	
	Paulo Henrique Dias	(19)99999-0008	
	Patrícia Fernandes	(19)99999-0009	
	Roberto Nunes	(19)99999-0010	
	Luciana Ribeiro	(19)99999-0011	
	André Luiz Barbosa	(19)99999-0012	
	Camila Freitas	(19)99999-0013	
	Bruno Teixeira	(19)99999-0014	

#EXERCÍCIO 23

-- Liste os produtos vendidos em cada pedido.

```
SELECT p.ped_numero, pr.pro_descricao
```

```
FROM pedido p
```

```
INNER JOIN item_pedido i
```

```
ON p.ped_numero = i.ped_numero
```

```
INNER JOIN produto pr
```

```
ON i.pro_codigo = pr.pro_codigo
```

```
ORDER BY p.ped_numero;
```

Result Grid			Filter Rows:
	ped_numero	pro_descricao	
▶	1	Arroz 5kg Tipo 1	
	1	Feijão Carioca 1kg	
	2	Macarrão Espaguete 500g	
	2	Óleo de Soja 900ml	
	3	Açúcar Refinado 1kg	
	3	Café Torrado e Moído 500g	
	4	Leite Integral 1L	
	4	Manteiga 200g	
	5	Pão de Forma 500g	
	5	Queijo Mussarela 1kg	
	6	Refrigerante Cola 2L	
	6	Presunto 1kg	
	7	Sabão em Pó 1kg	

#EXERCÍCIO 24

-- Mostre o número do pedido, o cliente e os produtos comprados.

```
SELECT p.ped_numero, c.cli_nome, pr.pro_descricao
```

```
FROM pedido p
```

```
INNER JOIN cliente c
```

```
ON p.cli_codigo = c.cli_codigo
```

```
INNER JOIN item_pedido i
```

```
ON p.ped_numero = i.ped_numero
```

```
INNER JOIN produto pr
```

```
ON i.pro_codigo = pr.pro_codigo
```

```
ORDER BY p.ped_numero;
```

	ped_numero	cli_nome	pro_descricao
▶	1	João Carlos Souza	Arroz 5kg Tipo 1
	1	João Carlos Souza	Feijão Carioca 1kg
	2	João Carlos Souza	Macarrão Espaguete 500g
	2	João Carlos Souza	Óleo de Soja 900ml
	3	Ana Paula Costa	Açúcar Refinado 1kg
	3	Ana Paula Costa	Café Torrado e Moído 500g
	4	Carlos Eduardo Lima	Leite Integral 1L
	4	Carlos Eduardo Lima	Manteiga 200g
	5	Fernanda Alves Rocha	Pão de Forma 500g
	5	Fernanda Alves Rocha	Queijo Mussarela 1kg
	6	Ricardo Gomes Pereira	Refrigerante Cola 2L
	6	Ricardo Gomes Pereira	Presunto 1kg
	7	Juliana Martins	Sabão em Pó 1kg
	7	Juliana Martins	Detergente Líquido 500ml

#EXERCÍCIO 25

-- Liste todos os clientes, mesmo aqueles que nunca fizeram pedidos.

```
SELECT c.cli_codigo, c.cli_nome, p.ped_numero
```

```
FROM cliente c
```

LEFT JOIN pedido p

ON c.cli_codigo = p.cli_codigo

ORDER BY c.cli_nome;

Result Grid			
Filter Rows:			
	cli_codigo	cli_nome	ped_numero
▶	15	Aline Castro	15
	3	Ana Paula Costa	3
	12	André Luiz Barbosa	12
	46	Beatriz Lima	31
	33	Beatriz Tavares	NULL
	14	Bruno Teixeira	14
	13	Camila Freitas	13
	44	Camila Souza	29
	31	Carla Nogueira	NULL
	4	Carlos Eduardo Lima	4
	39	Cristiane Lopes	NULL
	21	Daniela Souza	21
	35	Débora Rezende	NULL
	40	Diego Freitas	NULL

#EXERCÍCIO 26

-- Liste todos os produtos (descrição do produto) , mesmo aqueles que nunca foram vendidos.

SELECT pro_codigo, pro_descricao

FROM produto;

Result Grid		
Filter Rows:		
	pro_codigo	pro_descricao
▶	1	Arroz 5kg Tipo 1
	2	Feijão Carioca 1kg
	3	Macarrão Espaguete 500g
	4	Óleo de Soja 900ml
	5	Açúcar Refinado 1kg
	6	Café Torrado e Moído 500g
	7	Leite Integral 1L
	8	Manteiga 200g
	9	Pão de Forma 500g
	10	Queijo Mussarela 1kg
	11	Presunto 1kg
	12	Refrigerante Cola 2L
	13	Detergente Líquido 500ml
	14	Sabão em Pó 1kg

#EXERCÍCIO 27

- Liste todos os vendedores e seus respectivos pedidos,
- incluindo vendedores sem vendas.

```
SELECT v.ven_nome, p.ped_numero
```

```
FROM vendedor v
```

```
LEFT JOIN pedido p
```

```
ON v.ven_codigo = p.ven_codigo;
```

Result Grid   Filter Rows:		
	ven_nome	ped_numero
▶	Carlos Vendas	1
	Carlos Vendas	6
	Carlos Vendas	11
	Carlos Vendas	16
	Carlos Vendas	21
	Juliana Vendas	2
	Juliana Vendas	7
	Juliana Vendas	12
	Juliana Vendas	17
	Juliana Vendas	22
	Roberto Vendas	3
	Roberto Vendas	8
	Roberto Vendas	13
	Roberto Vendas	18

#EXERCÍCIO 28

- Liste todos os clientes e seus telefones,
- incluindo clientes sem telefone cadastrado.

```
SELECT c.cli_nome, t.tel_numero
```

```
FROM cliente c
```

```
LEFT JOIN telefone t
```

```
ON c.cli_codigo = t.cli_codigo;
```


Result Grid			Filter Rows:
	cli_nome	tel_numero	
▶	Maria Aparecida Silva	(19)99999-0001	
	João Carlos Souza	(19)99999-0002	
	Ana Paula Costa	(19)99999-0003	
	Carlos Eduardo Lima	(19)99999-0004	
	Fernanda Alves Rocha	(19)99999-0005	
	Ricardo Gomes Pereira	(19)99999-0006	
	Juliana Martins	(19)99999-0007	
	Paulo Henrique Dias	(19)99999-0008	
	Patrícia Fernandes	(19)99999-0009	
	Roberto Nunes	(19)99999-0010	
	Luciana Ribeiro	(19)99999-0011	
	André Luiz Barbosa	(19)99999-0012	
	Camila Freitas	(19)99999-0013	
	Bruno Teixeira	(19)99999-0014	

#EXERCÍCIO 29

-- Liste todos os pedidos e seus clientes,

-- incluindo pedidos sem cliente associado (caso existam).

```
SELECT p.ped_numero, c.cli_nome
```

```
FROM pedido p
```

```
LEFT JOIN cliente c
```

```
ON p.cli_codigo = c.cli_codigo;
```

Result Grid			Filter Rows:
	ped_numero	cli_nome	
▶	1	João Carlos Souza	
	2	João Carlos Souza	
	3	Ana Paula Costa	
	4	Carlos Eduardo Lima	
	5	Fernanda Alves Rocha	
	6	Ricardo Gomes Pereira	
	7	Juliana Martins	
	8	Paulo Henrique Dias	
	9	Patrícia Fernandes	
	10	Roberto Nunes	
	11	Luciana Ribeiro	
	12	André Luiz Barbosa	
	13	Camila Freitas	
	14	Bruno Teixeira	

#EXERCÍCIO 30

- Liste todos os itens de pedido e seus produtos,
- incluindo produtos sem vendas.

```
SELECT i.ped_numero, p.pro_descricao, i.item_quantidade
```

```
FROM item_pedido i
```

```
RIGHT JOIN produto p
```

```
ON i.pro_codigo = p.pro_codigo;
```

	ped_numero	pro_descricao	item_quantidade
▶	1	Arroz 5kg Tipo 1	1
	8	Arroz 5kg Tipo 1	1
	16	Arroz 5kg Tipo 1	1
	23	Arroz 5kg Tipo 1	1
	1	Feijão Carioca 1kg	1
	9	Feijão Carioca 1kg	1
	16	Feijão Carioca 1kg	1
	24	Feijão Carioca 1kg	1
	2	Macarrão Espaguete 500g	1
	9	Macarrão Espaguete 500g	1
	17	Macarrão Espaguete 500g	1
	24	Macarrão Espaguete 500g	1
	2	Óleo de Soja 900ml	1
	10	Óleo de Soja 900ml	1

#EXERCÍCIO 31

- Liste todos os clientes, todos os pedidos
- e os pagamentos realizados desses pedidos.

```
SELECT c.cli_nome, p.ped_numero, pg.pag_codigo, pg.pag_valor
```

```
FROM cliente c
```

```
LEFT JOIN pedido p
```

```
ON c.cli_codigo = p.cli_codigo
```

```
LEFT JOIN pagamento pg
```

```
ON p.ped_numero = pg.ped_numero;
```

Result Grid				
		Filter Rows:		Export:
	cli_nome	ped_numero	pag_codigo	pag_valor
▶	Maria Aparecida Silva	NULL	NULL	NULL
	João Carlos Souza	1	1	101.00
	João Carlos Souza	1	2	52.00
	João Carlos Souza	2	3	103.00
	João Carlos Souza	2	4	54.00
	Ana Paula Costa	3	5	105.00
	Ana Paula Costa	3	6	56.00
	Carlos Eduardo Lima	4	7	107.00
	Carlos Eduardo Lima	4	8	58.00
	Fernanda Alves Rocha	5	9	109.00
	Fernanda Alves Rocha	5	10	60.00
	Ricardo Gomes Pereira	6	11	111.00
	Ricardo Gomes Pereira	6	12	62.00
	Juliana Martins	7	13	113.00

#EXERCÍCIO 32

-- Liste todos os vendedores e todos os pedidos

-- utilizando UNION para representar FULL OUTER JOIN.

```
SELECT v.ven_nome, p.ped_numero
```

```
FROM vendedor v
```

```
LEFT JOIN pedido p
```

```
ON v.ven_codigo = p.ven_codigo
```

UNION

```
SELECT v.ven_nome, p.ped_numero
```

```
FROM vendedor v
```

```
RIGHT JOIN pedido p
```

```
ON v.ven_codigo = p.ven_codigo;
```

Result Grid			Filter Rows:
	ven_nome	ped_numero	
▶	Carlos Vendas	1	
	Carlos Vendas	6	
	Carlos Vendas	11	
	Carlos Vendas	16	
	Carlos Vendas	21	
	Juliana Vendas	2	
	Juliana Vendas	7	
	Juliana Vendas	12	
	Juliana Vendas	17	
	Juliana Vendas	22	
	Roberto Vendas	3	
	Roberto Vendas	8	
	Roberto Vendas	13	
	Roberto Vendas	18	

#EXERCÍCIO 33

-- Liste todos os produtos e todos os itens de pedido,

-- incluindo registros sem correspondência.

```
SELECT p.pro_descricao, i.ped_numero, i.item_quantidade
```

```
FROM produto p
```

```
LEFT JOIN item_pedido i
```

```
ON p.pro_codigo = i.pro_codigo
```

UNION

```
SELECT p.pro_descricao, i.ped_numero, i.item_quantidade
```

```
FROM produto p
```

```
RIGHT JOIN item_pedido i
```

```
ON p.pro_codigo = i.pro_codigo;
```

Result Grid   Filter Rows: Export			
	pro_descricao	ped_numero	item_quantidad
▶	Arroz 5kg Tipo 1	1	1
	Arroz 5kg Tipo 1	8	1
	Arroz 5kg Tipo 1	16	1
	Arroz 5kg Tipo 1	23	1
	Feijão Carioca 1kg	1	1
	Feijão Carioca 1kg	9	1
	Feijão Carioca 1kg	16	1
	Feijão Carioca 1kg	24	1
	Macarrão Espaguete 500g	2	1
	Macarrão Espaguete 500g	9	1
	Macarrão Espaguete 500g	17	1
	Macarrão Espaguete 500g	24	1
	Óleo de Soja 900ml	2	1
	Óleo de Soja 900ml	10	1

#EXERCÍCIO 34

-- Liste os clientes que nunca realizaram pedidos.

SELECT c.cli_nome

FROM cliente c

LEFT JOIN pedido p

ON c.cli_codigo = p.cli_codigo

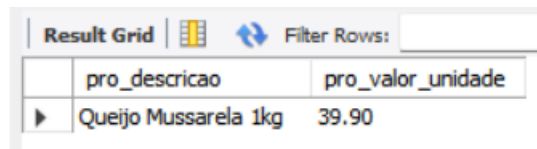
WHERE p.ped_numero IS NULL;

Result Grid   Filter Rows:	
	cli_nome
▶	Maria Aparecida Silva
	Thiago Batista
	Larissa Oliveira
	Rodrigo Martins
	Simone Carvalho
	Leandro Rocha
	Carla Nogueira
	Fábio Moreira
	Beatriz Tavares
	Márcio Pacheco
	Débora Rezende
	Sérgio Santana
	Paula Fernandes
	Marcelo Cunha

#EXERCÍCIO 35

-- Mostre o produto mais caro.

```
SELECT pro_descricao, pro_valor_unidade  
FROM produto  
WHERE pro_valor_unidade =  
(  
    SELECT MAX(pro_valor_unidade)  
    FROM produto  
);
```



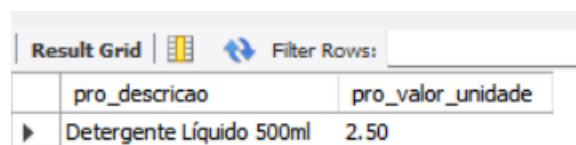
The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. The grid contains two columns: 'pro_descricao' and 'pro_valor_unidade'. The first row is expanded, showing 'Queijo Mussarela 1kg' with a value of 39.90.

	pro_descricao	pro_valor_unidade
▶	Queijo Mussarela 1kg	39.90

#EXERCÍCIO 36

-- Mostre o produto mais barato.

```
SELECT pro_descricao, pro_valor_unidade  
FROM produto  
WHERE pro_valor_unidade =  
(  
    SELECT MIN(pro_valor_unidade)  
    FROM produto  
);
```



The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. The grid contains two columns: 'pro_descricao' and 'pro_valor_unidade'. The first row is expanded, showing 'Detergente Líquido 500ml' with a value of 2.50.

	pro_descricao	pro_valor_unidade
▶	Detergente Líquido 500ml	2.50

#EXERCÍCIO 37

-- Liste os produtos com preço acima da média.

```
SELECT pro_descricao, pro_valor_unidade
```

```
FROM produto
```

```
WHERE pro_valor_unidade >
```

```
(
```

```
    SELECT AVG(pro_valor_unidade)
```

```
    FROM produto
```

```
);
```



The screenshot shows a 'Result Grid' window with a table of product data. The table has two columns: 'pro_descricao' and 'pro_valor_unidade'. There are 9 rows of data. The first row is 'Arroz 5kg Tipo 1' with a value of 29.90. The second row is 'Café Torrado e Moído 500g' with a value of 14.90. The third row is 'Queijo Mussarela 1kg' with a value of 39.90. The fourth row is 'Presunto 1kg' with a value of 29.90. The fifth row is 'Sabão em Pó 1kg' with a value of 12.90. The sixth row is 'Maionese 500g' with a value of 12.50. The seventh row is 'Granola 500g' with a value of 15.90. The eighth row is 'Cereal 300g' with a value of 13.90. The window also has a 'Filter Rows' button and a small icon of a grid.

pro_descricao	pro_valor_unidade
Arroz 5kg Tipo 1	29.90
Café Torrado e Moído 500g	14.90
Queijo Mussarela 1kg	39.90
Presunto 1kg	29.90
Sabão em Pó 1kg	12.90
Maionese 500g	12.50
Granola 500g	15.90
Cereal 300g	13.90

#EXERCÍCIO 38

-- Liste os vendedores com salário acima da média.

```
SELECT ven_nome, ven_salario_fixo
```

```
FROM vendedor
```

```
WHERE ven_salario_fixo >
```

```
(
```

```
    SELECT AVG(ven_salario_fixo)
```

```
    FROM vendedor
```

```
);
```

Result Grid			Filter Rows:
	ven_nome	ven_salario_fixo	
▶	Juliana Vendas	2500.00	
	Marcos Vendas	2300.00	
	Amanda Vendas	2400.00	
	Beatriz Vendas	2600.00	
	Gabriela Vendas	2500.00	
	Renata Vendas	2300.00	
	Larissa Vendas	2700.00	

#EXERCÍCIO 39

-- Mostre o cliente que realizou o maior número de pedidos.

```
SELECT c.cli_nome, COUNT(p.ped_numero) AS quantidade_pedidos
FROM cliente c
INNER JOIN pedido p
ON c.cli_codigo = p.cli_codigo
GROUP BY c.cli_codigo, c.cli_nome
ORDER BY quantidade_pedidos DESC
LIMIT 1;
```

Result Grid			Filter Rows:
	cli_nome	quantidade_pedidos	
▶	João Carlos Souza	2	

#EXERCÍCIO 40

-- Liste os vendedores que venderam mais pedidos

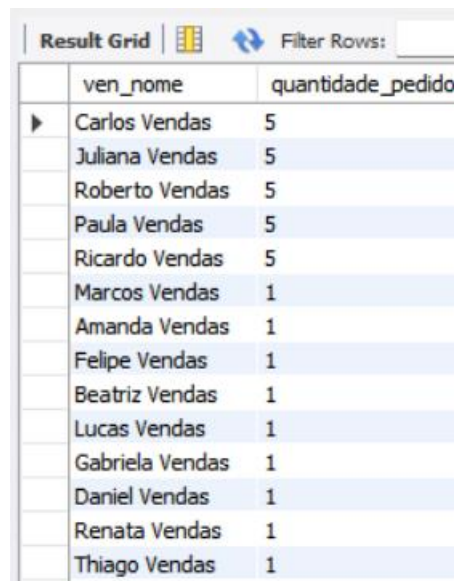
-- que os demais vendedores.

```
SELECT v.ven_nome, COUNT(p.ped_numero) AS quantidade_pedidos
FROM vendedor v
INNER JOIN pedido p
```


ON v.ven_codigo = p.ven_codigo

GROUP BY v.ven_codigo, v.ven_nome

ORDER BY quantidade_pedidos DESC;



The screenshot shows a 'Result Grid' window with a table containing two columns: 'ven_nome' and 'quantidade_pedido'. The table lists 15 salespeople and their respective number of orders, sorted in descending order. The first five salespeople (Carlos, Juliana, Roberto, Paula, and Ricardo) each have 5 orders, while the remaining ten each have 1 order.

ven_nome	quantidade_pedido
Carlos Vendas	5
Juliana Vendas	5
Roberto Vendas	5
Paula Vendas	5
Ricardo Vendas	5
Marcos Vendas	1
Amanda Vendas	1
Felipe Vendas	1
Beatriz Vendas	1
Lucas Vendas	1
Gabriela Vendas	1
Daniel Vendas	1
Renata Vendas	1
Thiago Vendas	1

#EXERCÍCIO 41

-- Quantos pedidos cada cliente realizou?

SELECT c.cli_nome, COUNT(p.ped_numero) AS quantidade_pedidos

FROM cliente c

LEFT JOIN pedido p

ON c.cli_codigo = p.cli_codigo

GROUP BY c.cli_codigo, c.cli_nome;

Result Grid			Filter Rows:
	cli_nome	quantidade_pedidos	
▶	Maria Aparecida Silva	0	
	João Carlos Souza	2	
	Ana Paula Costa	1	
	Carlos Eduardo Lima	1	
	Fernanda Alves Rocha	1	
	Ricardo Gomes Pereira	1	
	Juliana Martins	1	
	Paulo Henrique Dias	1	
	Patrícia Fernandes	1	
	Roberto Nunes	1	
	Luciana Ribeiro	1	
	André Luiz Barbosa	1	
	Camila Freitas	1	
	Bruno Teixeira	1	

#EXERCÍCIO 42

-- Quantos pedidos cada vendedor realizou?

```
SELECT v.ven_nome, COUNT(p.ped_numero) AS quantidade_pedidos
```

```
FROM vendedor v
```

```
LEFT JOIN pedido p
```

```
ON v.ven_codigo = p.ven_codigo
```

```
GROUP BY v.ven_codigo, v.ven_nome;
```

Result Grid			Filter Rows:
	ven_nome	quantidade_pedidos	
▶	Carlos Vendas	5	
	Juliana Vendas	5	
	Roberto Vendas	5	
	Paula Vendas	5	
	Ricardo Vendas	5	
	Marcos Vendas	1	
	Amanda Vendas	1	
	Felipe Vendas	1	
	Beatriz Vendas	1	
	Lucas Vendas	1	
	Gabriela Vendas	1	
	Daniel Vendas	1	
	Renata Vendas	1	
	Thiann Vendas	1	

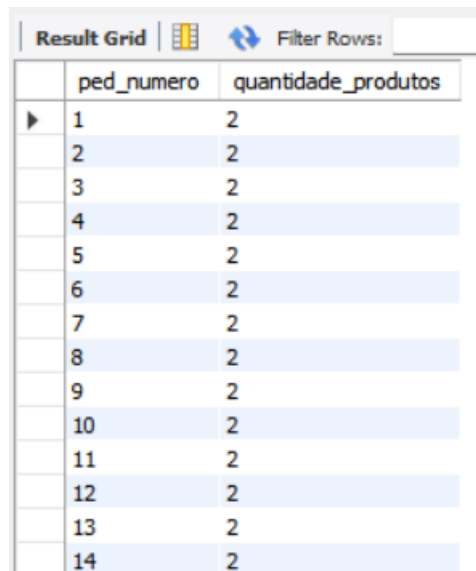
#EXERCÍCIO 43

-- Quantos produtos diferentes existem em cada pedido?

```
SELECT ped_numero, COUNT(pro_codigo) AS quantidade_produtos
```

```
FROM item_pedido
```

```
GROUP BY ped_numero;
```



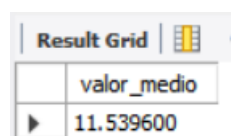
	ped_numero	quantidade_produtos
▶	1	2
	2	2
	3	2
	4	2
	5	2
	6	2
	7	2
	8	2
	9	2
	10	2
	11	2
	12	2
	13	2
	14	2

#EXERCÍCIO 44

-- Calcule o valor médio dos produtos.

```
SELECT AVG(pro_valor_unidade) AS valor_medio
```

```
FROM produto;
```



	valor_medio
▶	11.539600

#EXERCÍCIO 45

-- Mostre os clientes que realizaram mais de 2 pedidos.

```
SELECT c.cli_nome, COUNT(p.ped_numero) AS quantidade_pedidos
```

```

FROM cliente c

INNER JOIN pedido p

ON c.cli_codigo = p.cli_codigo

GROUP BY c.cli_codigo, c.cli_nome

HAVING COUNT(p.ped_numero) > 2;

```

Result Grid		Filter Rows:	
	cli_nome	quantidade_pedidos	

#EXERCÍCIO 46

-- Mostre os vendedores que realizaram mais de 5 vendas.

```

SELECT v.ven_nome, COUNT(p.ped_numero) AS quantidade_vendas

FROM vendedor v

INNER JOIN pedido p

ON v.ven_codigo = p.ven_codigo

GROUP BY v.ven_codigo, v.ven_nome

HAVING COUNT(p.ped_numero) > 5;

```

Result Grid		Filter Rows:	
	ven_nome	quantidade_vendas	

#EXERCÍCIO 47

-- Calcule o total vendido por produto.

```

SELECT p.pro_descricao,

```

```

SUM(i.item_quantidade * i.item_valor) AS total_vendido

FROM produto p

INNER JOIN item_pedido i

ON p.pro_codigo = i.pro_codigo

GROUP BY p.pro_codigo, p.pro_descricao;

```

Result Grid   Filter Rows:		
	pro_descricao	total_vendido
▶	Arroz 5kg Tipo 1	294.00
	Feijão Carioca 1kg	298.00
	Macarrão Espaguete 500g	302.00
	Óleo de Soja 900ml	306.00
	Açúcar Refinado 1kg	310.00
	Café Torrado e Moído 500g	213.00
	Leite Integral 1L	216.00
	Manteiga 200g	219.00
	Pão de Forma 500g	222.00
	Queijo Mussarela 1kg	225.00
	Presunto 1kg	228.00
	Refrigerante Cola 2L	231.00
	Detergente Líquido 500ml	234.00
	Sabão em Pó 1kg	237.00

#EXERCÍCIO 48

-- Calcule o total vendido por cliente.

```

SELECT c.cli_nome,

SUM(i.item_quantidade * i.item_valor) AS total_vendido

FROM cliente c

INNER JOIN pedido p

ON c.cli_codigo = p.cli_codigo

INNER JOIN item_pedido i

ON p.ped_numero = i.ped_numero

GROUP BY c.cli_codigo, c.cli_nome;

```

Result Grid			Filter Rows:
	di_nome	total_vend	
▶	João Carlos Souza	210.00	
	Ana Paula Costa	111.00	
	Carlos Eduardo Lima	115.00	
	Fernanda Alves Rocha	119.00	
	Ricardo Gomes Pereira	123.00	
	Juliana Martins	127.00	
	Paulo Henrique Dias	131.00	
	Patrícia Fernandes	135.00	
	Roberto Nunes	139.00	
	Luciana Ribeiro	143.00	
	André Luiz Barbosa	147.00	
	Camila Freitas	151.00	
	Bruno Teixeira	155.00	
	Aline Castro	159.00	

#EXERCÍCIO 49

-- Calcule a quantidade vendida por produto.

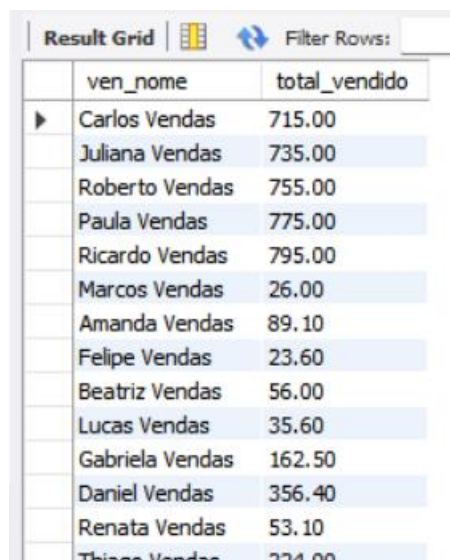
```
SELECT p.pro_descricao,
       SUM(i.item_quantidade) AS quantidade_vendida
FROM produto p
INNER JOIN item_pedido i
ON p.pro_codigo = i.pro_codigo
GROUP BY p.pro_codigo, p.pro_descricao;
```

Result Grid			Filter Rows:
	pro_descricao	quantidade_vendida	
▶	Arroz 5kg Tipo 1	4	
	Feijão Carioca 1kg	4	
	Macarrão Espaguete 500g	4	
	Óleo de Soja 900ml	4	
	Açúcar Refinado 1kg	4	
	Café Torrado e Moído 500g	3	
	Leite Integral 1L	3	
	Manteiga 200g	3	
	Pão de Forma 500g	3	
	Queijo Mussarela 1kg	3	
	Presunto 1kg	3	
	Refrigerante Cola 2L	3	
	Detergente Líquido 500ml	3	
	Sabão em Pó 1kg	3	

#EXERCÍCIO 50

-- Calcule o total vendido por vendedor.

```
SELECT v.ven_nome,  
  
       SUM(i.item_quantidade * i.item_valor) AS total_vendido  
  
FROM vendedor v  
  
INNER JOIN pedido p  
  
ON v.ven_codigo = p.ven_codigo  
  
INNER JOIN item_pedido i  
  
ON p.ped_numero = i.ped_numero  
  
GROUP BY v.ven_codigo, v.ven_nome;
```



The screenshot shows a 'Result Grid' window with a table containing two columns: 'ven_nome' and 'total_vendido'. The table lists 15 salespeople and their total sales. The first row is expanded, showing a small triangle icon to the left of the name 'Carlos Vendas'.

ven_nome	total_vendido
Carlos Vendas	715.00
Juliana Vendas	735.00
Roberto Vendas	755.00
Paula Vendas	775.00
Ricardo Vendas	795.00
Marcos Vendas	26.00
Amanda Vendas	89.10
Felipe Vendas	23.60
Beatriz Vendas	56.00
Lucas Vendas	35.60
Gabriela Vendas	162.50
Daniel Vendas	356.40
Renata Vendas	53.10
Thiago Vendas	224.00

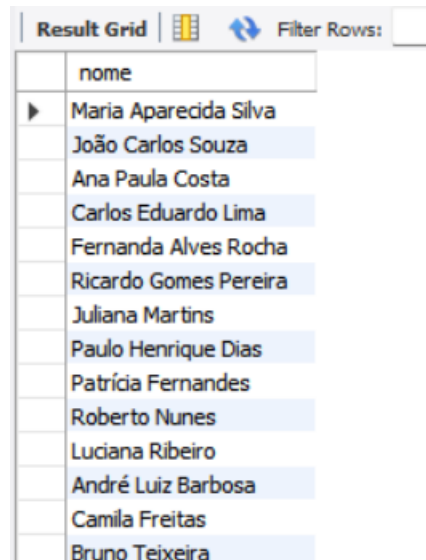
#EXERCÍCIO 51

-- Liste todos os nomes de clientes e vendedores sem repetição.

```
SELECT cli_nome AS nome  
  
FROM cliente  
  
  
  
UNION
```

```
SELECT ven_nome AS nome
```

```
FROM vendedor;
```



The screenshot shows a 'Result Grid' window with a 'Filter Rows' button. The grid contains a single column named 'nome' with 15 rows of names. The first row is 'Maria Aparecida Silva', and the last row is 'Bruno Teixeira'. The rows are alternatingly highlighted in white and light blue.

nome
Maria Aparecida Silva
João Carlos Souza
Ana Paula Costa
Carlos Eduardo Lima
Fernanda Alves Rocha
Ricardo Gomes Pereira
Juliana Martins
Paulo Henrique Dias
Patrícia Fernandes
Roberto Nunes
Luciana Ribeiro
André Luiz Barbosa
Camila Freitas
Bruno Teixeira

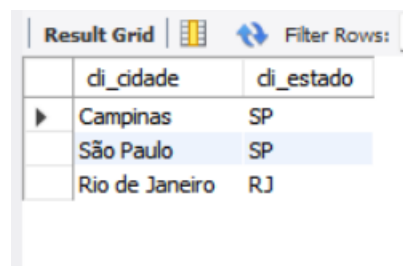
#EXERCÍCIO 52

-- Liste cidades dos clientes e estados sem repetição.

```
SELECT cli_cidade, cli_estado
```

```
FROM cliente
```

```
GROUP BY cli_cidade, cli_estado;
```



The screenshot shows a 'Result Grid' window with a 'Filter Rows' button. The grid contains two columns: 'cli_cidade' and 'cli_estado'. There are three rows of data: 'Campinas' and 'SP', 'São Paulo' and 'SP', and 'Rio de Janeiro' and 'RJ'. The first row is highlighted in light blue.

cli_cidade	cli_estado
Campinas	SP
São Paulo	SP
Rio de Janeiro	RJ

#EXERCÍCIO 53

-- Liste todos os nomes de clientes e vendedores

-- mantendo repetições.

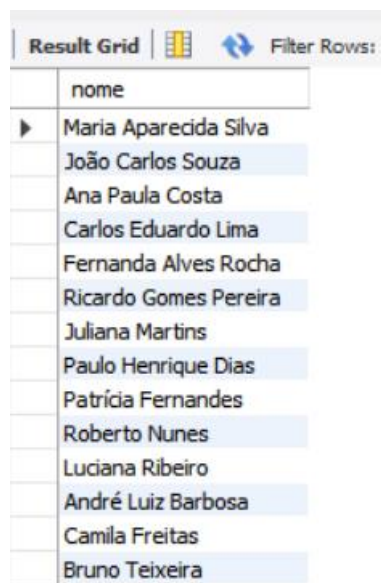
```
SELECT cli_nome AS nome
```

```
FROM cliente
```

```
UNION ALL
```

```
SELECT ven_nome AS nome
```

```
FROM vendedor;
```



The image shows a screenshot of a database application's 'Result Grid'. The grid has a single column titled 'nome'. It contains 15 rows of names, alternating between light blue and white background colors. The names listed are: Maria Aparecida Silva, João Carlos Souza, Ana Paula Costa, Carlos Eduardo Lima, Fernanda Alves Rocha, Ricardo Gomes Pereira, Juliana Martins, Paulo Henrique Dias, Patrícia Fernandes, Roberto Nunes, Luciana Ribeiro, André Luiz Barbosa, Camila Freitas, and Bruno Teixeira. Above the grid, there are icons for 'Result Grid', a grid icon, a refresh icon, and a 'Filter Rows:' label.

nome
Maria Aparecida Silva
João Carlos Souza
Ana Paula Costa
Carlos Eduardo Lima
Fernanda Alves Rocha
Ricardo Gomes Pereira
Juliana Martins
Paulo Henrique Dias
Patrícia Fernandes
Roberto Nunes
Luciana Ribeiro
André Luiz Barbosa
Camila Freitas
Bruno Teixeira

#EXERCÍCIO 54

-- Liste nomes que existam tanto em CLIENTE quanto em VENDEDOR.

```
SELECT cli_nome AS nome
```

```
FROM cliente
```

```
INNER JOIN vendedor
```

```
ON cliente.cli_nome = vendedor.ven_nome;
```

Result Grid		Filter Rows:
	nome	

#EXERCÍCIO 55

-- Liste vendedores que nunca participaram de vendas.

SELECT v.ven_nome

FROM vendedor v

LEFT JOIN pedido p

ON v.ven_codigo = p.ven_codigo

WHERE p.ped_numero IS NULL;

Result Grid		Filter Rows:
	ven_nome	